



Unifi Vault

Security Review

unifi

Disclaimer

Security Review

Unifi Vault



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

Unifi Vault

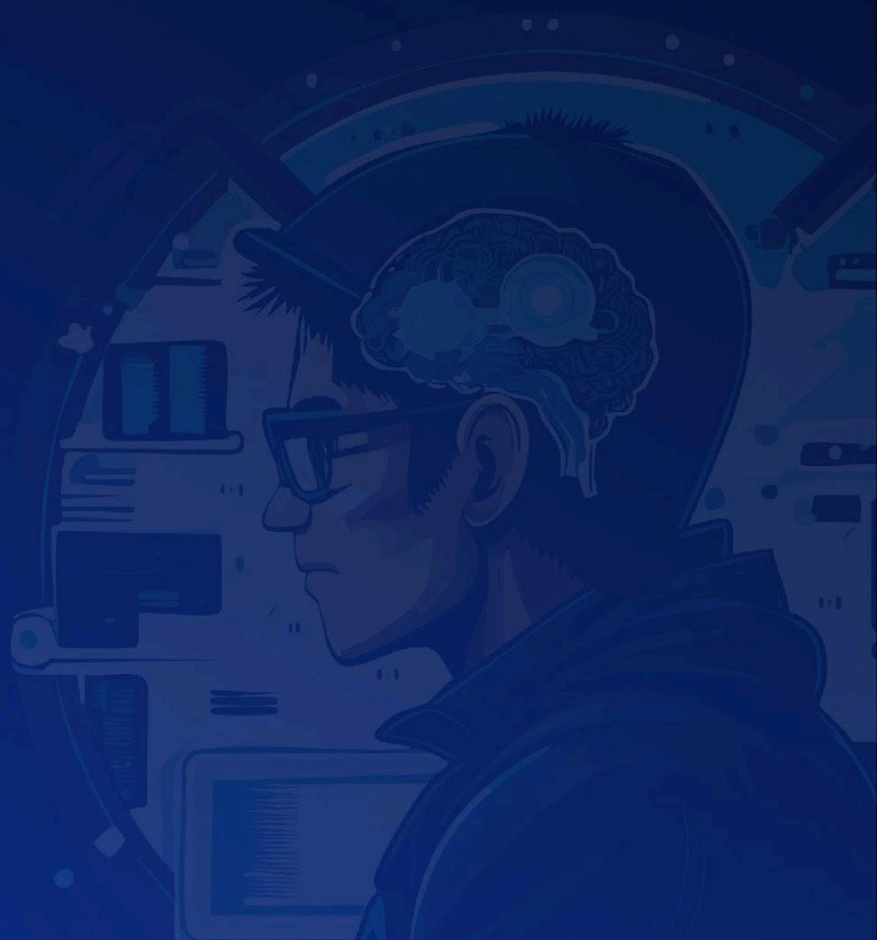


Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S-Vault-H01	18
3S-Vault-H02	21
3S-Vault-H03	23
3S-Vault-H04	25
3S-Vault-H05	27
3S-Vault-M01	29
3S-Vault-L01	32
3S-Vault-L02	33
3S-Vault-L03	35
3S-Vault-N01	37

Summary Security Review

Unifi Vault



Summary

Three Sigma audited Unifi in a 4 person week engagement. The audit was conducted from 04/03/2026 to 18/03/2026.

Protocol Description

UniFi Contract is a single-asset DeFi vault system built in Solidity (Foundry) and deployed on the KAIA blockchain, featuring a role-based Vault that accepts user deposits, dynamically allocates capital across pluggable yield strategies (such as SuperEarnStrategy, which routes funds through a SuperEarn Router into a CooldownVault), and uses an EMA-based liquidity buffer mechanism to automatically rebalance between vault liquidity and strategy investments; the protocol includes a SecurityAgent deposit router, EIP-712 signed withdrawal permits, strategy migration support, and a comprehensive governance structure with distinct roles for governance, manager, guardian, and inspector.

Scope Security Review

Unifi Vault



Scope

Filepath	nSLOC
contracts/vault/Vault.sol	1231
contracts/strategies/SuperEarnStrategy.sol	543
contracts/vault/VaultRoles.sol	314
contracts/vault/SecurityAgent.sol	96
contracts/libs/TwabLib.sol	86
contracts/Errors.sol	52
contracts/libs/FixedPointMath.sol	51
contracts/vault/VaultCompatibilityCodes.sol	19
Total	2392

Methodology Security Review

Unifi Vault



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard Security Review

Unifi Vault



Project Dashboard

Application Summary

Name	Unifi
Repository	https://github.com/unifi-project-ext/vault-contract
Commit	378c563
Language	Solidity
Platform	Kaia

Engagement Summary

Timeline	04/03/2026 to 18/03/2026
Nº of Auditors	2
Review Time	4 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	5	5	0
Medium	1	1	0
Low	3	2	1
None	1	1	0

Category Breakdown

Suggestion	1
Documentation	0
Bug	9
Optimization	0
Good Code Practices	0

Risk Section Security Review

Unifi Vault



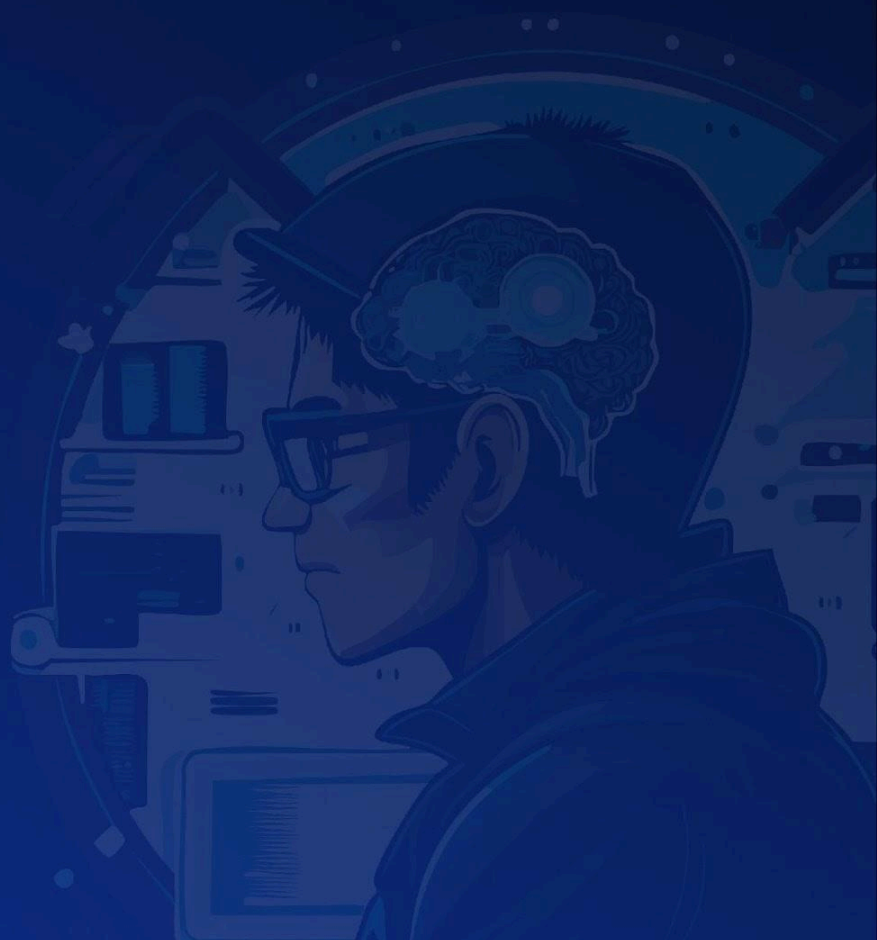
Risk Section

No risks were identified.

Findings

Security Review

Unifi Vault



Findings

3S-Vault-H01

Old Strategy's Vault Pointer Not Updated After Dirty Strategy Migration + Vault Migration

Id	3S-Vault-H01
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #c2eb8d9 .

Summary

When a vault migration follows a dirty strategy migration, pending redemptions referencing the old strategy become permanently unclaimable because the old strategy's unifiVault pointer is never updated to the new vault.

Description

Initial State

Contracts and their pointers:

Contract | Key State

-- | --

Vault V1 | strategy → Strategy A, _pendingRedemptions: [{Strategy A, req#1, 10k}, {Strategy A, req#2, 5k}]

Strategy A | unifiVault → V1, _yShares = 100k

CooldownVault | holds req#1 (10k), req#2 (5k)

Everything works. V1 calls Strategy A, Strategy A checks `msg.sender == unifiVault (V1)` — passes.

Step 1: Dirty Strategy Migration

Manager calls **V1.migrateStrategy(StrategyB)**. Shares and assets move to Strategy B, but pending redemptions still reference Strategy A.

Contract | Key State After

-- | --

Vault V1 | strategy → Strategy B, _pendingRedemptions: [{Strategy A, req#1, 10k}, {Strategy A, req#2, 5k}]

Strategy B (new) | unifiVault → V1, _yShares = 100k

Strategy A (old) | unifiVault → V1 ← still points to V1, _yShares = 0

CooldownVault | req#1 (10k), req#2 (5k) — waiting for claim

This is fine. V1 can still call Strategy A.claimWithdrawal() because Strategy A's unifiVault still equals V1.

Step 2: Vault Migration Proposed & Accepted

Manager calls **V1.proposeVault(V2)**, then **V2.acceptVaultFrom(V1)**.

What acceptVaultFrom does:

1. Imports **_pendingRedemptions** → copies [Strategy A, req#1] and [Strategy A, req#2] into V2
2. Imports EMA state
3. Sets **V2.strategy = Strategy B** (reads V1.strategy() which is B)
4. Calls **V1.acceptMigrationRequest()** → V1 transfers all USDT to V2, clears its own pending list
5. Calls **SecurityAgent.acceptVault()** → SA now points to V2
6. Calls **Strategy B.acceptVault()** → Strategy B now points to V2
7. Never touches Strategy A — its unifiVault remains V1

Contract | Key State After

-- | --

Vault V2 (new) | strategy → Strategy B, _pendingRedemptions: [{Strategy A, req#1, 10k}, {Strategy A, req#2, 5k}]

Vault V1 (old) | migrationState = Accepted, effectively dead, _pendingRedemptions: []

Strategy B | unifiVault → V2 - updated

Strategy A | unifiVault → V1 - STALE — never updated

Step 3: The Permanent Revert

Someone calls **V2.claimRedemptions(0)** to process the pending queue:

V2 reads `_pendingRedemptions[0] = {strategy: Strategy A, reqId: 1}`

V2 calls → `Strategy A.claimWithdrawal(1)`

Strategy A's onlyVault modifier checks: `msg.sender(V2) == unifiVault(V1) ? NO → REVERT`

V2's catch block: skip, increment i

V2 reads `_pendingRedemptions[1] = {strategy: Strategy A, reqId: 2}`

Same revert. Same skip.

Result: 0 processed. Both entries remain in array forever.

Impact

Funds locked in the CooldownVault with no recovery path. The old strategy can't claim (vault pointer wrong), the new vault can't claim (access denied), and the entries can't be evicted from the queue.

Recommendation

During strategy migration, update or remove all pending redemptions referencing the old strategy. Or, during vault migration, update old strategy's vault pointer.

3S-Vault-H02

No Asset Compatibility Check in `acceptVaultFrom`

Id	3S-Vault-H02
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #69f4a8d .

Summary

`acceptVaultFrom()` imports strategy, pending redemptions, and liquidity from an old vault without verifying that the old vault manages the same underlying asset.

Description

When a new vault accepts a migration via `acceptVaultFrom()`, it performs several compatibility checks — confirming the new vault was proposed, no settlement is pending, and the old vault supports the expected interface. However, it never verifies that `oldVault.asset() == this.asset`.

Compare with `setStrategy()`, which explicitly validates asset compatibility:

```
if (IStrategy(newStrategy).asset() != asset) revert InvalidAddress();
```

No equivalent check exists in `acceptVaultFrom()`. As a result:

- The strategy pointer is imported even if the strategy operates on a different token
- Pending redemptions denominated in one asset are imported into a vault tracking a different asset
- Liquidity is transferred and credited to `_expectedBalance` without verifying token identity

Impact

If a manager mistakenly proposes a vault with a mismatched asset (e.g., USDC vault migrating into a USDT vault), the migration will succeed silently. The new vault's accounting

(`_expectedBalance`, `_totalPendingRedemptionAmount`, rebalance calculations) will be corrupted, and subsequent operations like `repay()`, `rebalance()`, and `claimRedemptions()` will operate on incorrect assumptions about what token the strategy holds.

Recommendation

Add an asset compatibility check at the start of `acceptVaultFrom()`:

```
(success, data) = oldVault.staticcall(abi.encodeWithSelector(bytes4(keccak256("asset()"))));  
if (!success) revert VaultIncompatible(uint8(VaultCompatibilityCode.NoAsset));  
address oldAsset = abi.decode(data, (address));  
if (oldAsset != asset) revert  
VaultIncompatible(uint8(VaultCompatibilityCode.AssetMismatch));
```

3S-Vault-H03

Permanent Denial-of-Service via Blocked/Blacklisted recipient Address in `claimSettlePayment`

Id	3S-Vault-H03
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #2d026f1 .

Summary

The `settlePayment()` function allows the `permit.to` (recipient) address to be any non-zero address, including USDT-blacklisted addresses or contracts that revert on token receipt. Once a settlement is queued with such a recipient, `claimSettlePayment()` permanently reverts on the `safeTransfer` to that address. Since `settlementCooldownRequestId` is only cleared within the same atomic transaction as the transfer, and there is no cancel mechanism, the settlement becomes permanently stuck. This blocks all downstream operations that check for pending settlements, effectively bricking the vault.

Description

The `settlePayment()` function accepts any non-zero address as the recipient. This address is stored in `_pendingSettlement` and a cooldown period begins. Once the cooldown elapses, anyone can call `claimSettlePayment()`, which claims funds from the strategy and then calls `safeTransfer` to the stored recipient. If that transfer reverts — because the recipient intentionally provided a blacklisted address or was blacklisted by USDT during the cooldown, or was a contract that refuses incoming tokens — the entire transaction rolls back, including the state cleanup that was supposed to mark the settlement as complete. Every future attempt to call `claimSettlePayment()` hits the same revert, creating an infinite loop with no escape. There is no admin cancel function, no override, and even vault migration checks for pending settlements and blocks itself.

Impact

The stuck settlement permanently blocks six vault operations: new settlements, claiming the current settlement, changing the claimer role, setting a new strategy, migrating the strategy, and migrating to a new vault. The vault can still process repayments and rebalance, but it loses the ability to settle payments or upgrade in any way. The settlement funds remain locked forever — the strategy won't release them because the claim keeps reverting, and no one can cancel the request. The vault is effectively bricked for all administrative and settlement functions with no recovery path.

Recommendation

Add a manager-gated **cancelSettlement()** function that clears **settlementCooldownRequestId** and **_pendingSettlement** without attempting the transfer, returning funds to the vault's general liquidity. Alternatively, wrap the **safeTransfer** in **claimSettlePayment()** with a try/catch so that if the transfer to the recipient fails, the funds are sent to the manager as a fallback, ensuring the settlement always completes and the vault is never permanently stuck.

3S-Vault-H04

rebalance function double-counts yesterday's volume in EMA calculation

Id	3S-Vault-H04
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #5ba3559 .

Summary

The **rebalance()** function applies yesterday's volume to the EMA formula a second time after **_updateEmaForPastDays()** has already incorporated it, causing **emaForDecision** to over-weight yesterday's volume.

Description

_updateEmaForPastDays() iterates from **lastEmaUpdateDay** up to and including **today - 1** (yesterday), folding each day's volume into **emaOfRepayments**:

```
// _updateEmaForPastDays — last iteration always processes yesterday
uint256 dayToProcess = lastEmaUpdateDay + i; // final i = today - 1
emaOfRepayments = ((volumeForDay * smoothingFactor) +
    (emaOfRepayments * (1e18 - smoothingFactor))) / 1e18;
```

Immediately after calling this function, **rebalance()** applies yesterday's volume again:

```
_updateEmaForPastDays(); // emaOfRepayments already includes yesterday
uint256 yesterdaysVolume = repaymentVolumePerDay[today - 1];
uint256 emaForDecision = ((yesterdaysVolume * smoothingFactor) + // ← applied twice
    (emaOfRepayments * (1e18 - smoothingFactor))) / 1e18;
```

At this point **emaOfRepayments** is a fully up-to-date EMA including yesterday. Feeding yesterday's volume into the formula again treats it as if it occurred twice, inflating or deflating the EMA.

Recommendation

Remove the redundant inline EMA calculation and use **emaOfRepayments** directly, since **_updateEmaForPastDays()** has already produced the correct up-to-date value:

```
_updateEmaForPastDays();
- uint256 today = block.timestamp / 1 days;
- uint256 yesterdaysVolume = today == 0
-   ? 0
-   : repaymentVolumePerDay[today - 1];
-
- uint256 emaForDecision = ((yesterdaysVolume * smoothingFactor) +
-   (emaOfRepayments * (1e18 - smoothingFactor))) / 1e18;
+ uint256 emaForDecision = emaOfRepayments;
```

3S-Vault-H05

silent **reportStrategy()** failure during emergency redemption can permanently lock funds in strategy

Id	3S-Vault-H05
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #51d1458 .

Summary

Silent failure of the **vault.reportStrategy()** call in **_executeEmergencyRedemption()** can leave the strategy in a locked state where **_unclaimedCount > 0**, preventing **withdrawIdleToVault()** from executing and trapping funds in the strategy until a vault migration is done.

Description

The **SuperEarnStrategy** implements a circuit breaker that automatically triggers an emergency redemption when the external **yVault** share price drops below a configured threshold. This mechanism is executed through **monitorAndProtect()**, which calls **_executeEmergencyRedemption()** once the threshold condition is met.

During this emergency flow, the strategy:

1. Redeems all **yShares** through the router.
2. Creates an internal withdrawal request by incrementing **_unclaimedCount** and storing a **WithdrawalRequest**.
3. Pauses deposits and updates accounting.
4. Attempts to notify the vault by calling **reportStrategy()** with the **EmergencyRedemptionRequested** report type.

The vault uses this report to append a new entry into its **_pendingRedemptions** queue, allowing it to track the withdrawal lifecycle.

However, the call to **reportStrategy()** is wrapped inside a **try/catch** block:

```
try IVault(unifiVault).reportStrategy(...) { }
catch (bytes memory reason) {
    emit ReportStrategyFailed(reqId, reason);
}
```

If the call to **reportStrategy()** reverts for any reason , the strategy silently avoids the error thinking it can send funds to vault using **withdrawIdleToVault** function.

At this point:

- The strategy has already redeemed all **yShares**.
- **_unclaimedCount** has been incremented.
- The internal withdrawal request exists in the strategy.
- The vault has not recorded the pending redemption.

This creates a critical accounting desynchronization between the strategy and the vault.

The strategy relies on **_unclaimedCount** reaching zero before allowing idle assets to be returned to the vault via **withdrawIdleToVault()**. Because the vault never recorded the redemption request, the normal redemption lifecycle cannot complete, meaning **_unclaimedCount** can never be decremented.

As a result, **withdrawIdleToVault()** remains permanently blocked, leaving the redeemed assets stuck in the strategy contract.

The only remaining recovery option is to claim directly on super earn directly and perform a vault migration to recover the stuck funds.

Recommendation

Ensure emergency redemption flows include a reliable recovery mechanism so failed vault notifications cannot leave the strategy and vault in a permanently desynchronized, fund-locking state.

3S-Vault-M01

Temporary DoS in claimRedemptions due to swap-and-pop logic

Id	3S-Vault-M01
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #c4fb549 .

Summary

The **claimRedemptions** function contains a critical logic flaw caused by combining a forward-iterating loop, a strict **maxIterations** cap, and a swap-and-pop array removal mechanism. When processing ready requests, unready requests from the end of the array are systematically swapped to the front. Because the loop caps out at 50 iterations, these unready items form an impenetrable wall at the start of the array, permanently blocking access to valid, processable requests stuck further down the list. This effectively soft-locks the redemption process for those users until the front items become ready.

Description

When a request is successfully processed, **_removePendingRedemptionAt(i)** replaces the current item with the last item in the array and pops the end. Because **i** is **not incremented** on success, the loop immediately evaluates this newly swapped item. If it is not ready, **i** increments, leaving the unready item stuck at the front of the array.

Over multiple iterations, this logic systematically vacuums unready items from the back of the array and deposits them at the front, eventually consuming the entire **maxIterations** allowance.

Here is how the blockade occurs using a scenario with 150 requests, where the first 100 are ready and the last 50 are not, with **maxIterations = 50**.

1. Initial State

R1 through R100 are ready to process. R101 through R150 are not ready.

_pendingRedemptions (150 total items):

Index: 0 1 2 ... 49 50 ... 99 100 101 ... 149

Request: R1 R2 R3 ... R50 R51 ... R100 R101 R102 ... R150

State: [===== READY (100) =====] [= NOT READY (50) =]

2. Loop Tick 1 — **i = 0**

The loop evaluates index 0 (R1), which is ready.

- R1 is processed and **_removePendingRedemptionAt(0)** is called.
- The last item (R150) is swapped into index 0.
- **i** remains 0.

_pendingRedemptions (149 total items):

Index: 0 1 2 ... 49 50 ... 99 100 101 ... 148

Request: R150* R2 R3 ... R50 R51 ... R100 R101 R102 ... R149

^

R1 was processed. R150 swapped to front.

3. Loop Tick 2 — **i = 0** (again)

The loop evaluates index 0 again, which is now R150.

- R150 is not ready — the **eta** guard fires (or **claimWithdrawal** reverts and the **catch** block fires).
- The item is skipped and **i** is incremented to 1.

4. Loop Tick 3 — **i = 1**

The loop evaluates index 1 (R2), which is ready.

- R2 is processed. The new last item (R149) is swapped into index 1 and the end is popped.
- **i** remains 1.
- The next tick evaluates R149, finds it not ready, and increments **i** to 2.

5. The Permanent Blockade — fast forward to **i = 50**

Every time the loop finds a ready item it drags an unready item from the back to replace it, then increments **i** to skip past it. Each successful claim therefore costs **two ticks**: one for the claim, one for the forced skip of the swapped-in tail element.

By the time **i** hits 50, the contract has successfully processed 50 ready items, but indices 0 through 49 are now entirely filled with unready items.

_pendingRedemptions (100 total items remaining):

Index: 0 1 2 ... 49 50 51 ... 99

Request: R150 R149 R148 ... R101 R51 R52 ... R100

State: [===== NOT READY (50) =====] [=== READY (50) ===]

At this exact moment **i** equals 50, which matches **maxIterations**. The loop exits.

The Impact

On any subsequent call to **claimRedemptions()**, **i** starts at 0. The loop checks indices 0 through 49, finds that all 50 items (R150 down to R101) are not ready, and increments **i** each time. Once **i** hits 50 it reaches the **maxIterations** cap and exits.

Call 2:

i=0 : R150 not ready → skip, **i=1**

i=1 : R149 not ready → skip, **i=2**

...

i=49 : R101 not ready → skip, **i=50**

^^^^

i=50 >= **maxIterations=50** → LOOP EXITS. 0 claims made.

The 50 ready items (R51 to R100) sitting at indices 50 through 99 are perfectly valid and processable, but the loop can never reach them. They are permanently locked until all items at the front of the array become ready.

Recommendation

Update the **claimRedemptions** function to enable redemptions from specified starting index

3S-Vault-L01

MinRebalanceInterval can be set to zero

Id	3S-Vault-L01
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #bfa382e .

Description

setMinRebalanceInterval() lets the manager configure the minimum time between **rebalance()** calls. The **rebalance()** function moves USDT between the vault and strategy to maintain optimal liquidity based on the EMA of daily repayment volumes. The interval setter has no lower-bound validation — it accepts any uint256 including zero, which would allow the rebalancer to call **rebalance()** every block.

Impact

Allows spam rebalancing every block. Slippage accumulation, MEV extraction, gas griefing. Requires compromised/negligent manager.

Recommendation

Introduce lower bound check

3S-Vault-L02

Inspector Separation Not Enforced in Code

Id	3S-Vault-L02
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Acknowledged

Summary

SECURITY.md mandates that borrow inspectors and repay inspectors must be different addresses, but `_addInspector()` and `_addRepayInspector()` in **VaultRoles.sol** perform no cross-list validation, allowing the manager to assign both roles to the same address.

Description

The protocol's security model relies on a dual-authorization design: borrow inspectors control fund inflows (authorizing deposits via `SecurityAgent`), while repay inspectors control fund outflows (signing EIP-712 repay/settle permits).

SECURITY.md explicitly requires these to be separate addresses:

"배포 시 BORROW_INSPECTOR와 REPAY_INSPECTOR를 서로 다른 주소로 관리하고"

("At deployment, manage BORROW_INSPECTOR and REPAY_INSPECTOR as different addresses")

However, `_addInspector()` and `_addRepayInspector()` each only validate against `address(0)` and duplicates within their own list. Neither checks whether the address already exists in the other role's list:

```
function _addInspector(address newInspector) internal {
    if (newInspector == address(0)) revert InvalidAddress();
    if (!_isInspector[newInspector]) { // ← only checks own list
        _isInspector[newInspector] = true;
        ...
    }
}
```

```
function _addRepayInspector(address newInspector) internal {
    if (newInspector == address(0)) revert InvalidAddress();
    if (!_isRepayInspector[newInspector]) { // ← only checks own list
        _isRepayInspector[newInspector] = true;
        ...
    }
}
```

Impact

If the same address is assigned both roles — whether by mistake or by a compromised manager — the two-key security model collapses into a single-key model.

Recommendation

Add cross-list validation in both functions:

```
function _addInspector(address newInspector) internal {
    if (newInspector == address(0)) revert InvalidAddress();
    + if (_isRepayInspector[newInspector]) revert InvalidAddress();
    ...
}
function _addRepayInspector(address newInspector) internal {
    if (newInspector == address(0)) revert InvalidAddress();
    + if (_isInspector[newInspector]) revert InvalidAddress();
    ...
}
```

3S-Vault-L03

Unbounded Loop in `_importEmaStateFrom()` Can Cause Out-of-Gas During Vault Migration

Id	3S-Vault-L03
Classification	Low
Impact	Low
Likelihood	
Category	Bug
Status	Addressed in #e508cd6 .

Summary

The NatSpec comment on `_importEmaStateFrom()` states the history copy is capped at 30 days, but no such cap is enforced in the implementation. The loop iterates from `oldLastEmaUpdateDay` all the way to `today`, issuing one `staticcall` per day. If the old vault's EMA was last updated months or years ago, the unbounded iteration will exhaust the block gas limit, permanently bricking vault migration.

Description

The comment explicitly claims a 30-day maximum:

```
// 2. Calculate history data range (based on current vault's emaPeriod, maximum 30 days)
uint256 startDay = oldLastEmaUpdateDay < (today - 1)
    ? oldLastEmaUpdateDay
    : (today - 1);
```

However, `startDay` is set to `oldLastEmaUpdateDay` unconditionally when it is less than `today - 1`. There is no subtraction of 30 or clamping relative to `today`. The loop then runs without any bound:

```
for (uint256 day = startDay; day <= today; day++) {
```

```

(success, data) = oldVault.staticcall(    // one external call per day
    abi.encodeWithSelector(REPAYMENT_VOLUME_SELECTOR, day)
);
...
}

```

Each iteration performs an external **staticcall**, which is significantly more expensive than a simple storage read. If **oldLastEmaUpdateDay** is 180 days in the past, the loop executes 180 **staticcalls**. At ~2,100 gas per **staticcall** overhead plus calldata and return data decoding, this can easily exceed the block gas limit.

The impact is critical because **_importEmaStateFrom()** is called inside **acceptVaultFrom()**, which is the only entry point for completing a vault migration. A revert here means the migration cannot be done at all

Contrast with **_updateEmaForPastDays()**, which correctly enforces the 30-day cap:

```

// _updateEmaForPastDays — correctly capped
uint256 maxDaysToProcess = 30;
if (daysToProcess > maxDaysToProcess) {
    lastEmaUpdateDay = today - maxDaysToProcess;
    daysToProcess = maxDaysToProcess;
}

```

The same guard was intended for **_importEmaStateFrom()** per its own comment but was never implemented.

Recommendation

Apply the same 30-day cap that **_updateEmaForPastDays()** already uses.

3S-Vault-N01

Informationals

Id	3S-Vault-N01
Classification	None
Category	Suggestion
Status	Addressed in #c5bc53e .

I-01: Peak Share Price Never Resets — Circuit Breaker Re-Trigger Loop

Description: Once **peakSharePrice** is set during an emergency redemption, it never resets. After recovery, the share price must exceed the old peak before the circuit breaker can trigger again, creating a window where real drops go undetected.

Recommendation: Reset **peakSharePrice** after successful recovery so the circuit breaker recalibrates from the current price.

I-02: `requestWithdrawal` Uses `previewDeposit` Instead of `previewWithdraw`

Description: `requestWithdrawal` calls `previewDeposit` to estimate shares for a given asset amount. The correct function is `previewWithdraw`, which accounts for withdrawal-specific fees/slippage in ERC-4626 vaults.

Recommendation: Replace `previewDeposit` with `previewWithdraw`.

I-03: `maxLossBps` Unit Mismatch with `CooldownVault.claim()`

Description: **maxLossBps** is stored in basis points (10000 = 100%) but `CooldownVault.claim()` expects **maxLoss** in a different unit scale. This mismatch could cause unexpected revert or incorrect loss tolerance.

Recommendation: Verify the unit convention of `CooldownVault.claim()`'s **maxLoss** parameter and convert **maxLossBps** accordingly before passing it.

I-04: No Zero-Address Check on `permit.to` in `repay()`

Description: The `repay` function accepts a `to` address from the permit signature without validating it's non-zero. Funds would be transferred to **address(0)** if a malformed permit is submitted.

Recommendation: Add `require(permit.to != address(0))` before processing the repayment.

I-05: `migrationAccepted` State Variable Is Redundant

Description: The `migrationAccepted` flag duplicates information already derivable from other state (e.g., checking if migration has been processed). It adds storage cost and maintenance burden without providing unique functionality.

Recommendation: Remove `migrationAccepted` and derive the migration status from existing state.

I-06: Unnecessary Manual Array Copy in `getPendingRedemptionRequests`

Description: The function manually copies `_pendingRedemptions` into a new array element-by-element. For memory return types, the Solidity compiler already handles the copy automatically.

Recommendation: Replace the manual loop with `return _pendingRedemptions`.

I-05: Missing Events on Admin State Changes

Description: Several admin functions modify critical state without emitting events:

Contract | Function | Missing Event

Vault | `setMaxPendingRedemptions` | Queue size change

Vault | `recoverWrongToken` | Token recovery

SuperEarnStrategy | `proposeVault` (cancel path) | Proposal cancellation

Vault | `invalidateNonce` | Nonce invalidation

Recommendation: Add event emissions to each function for off-chain monitoring and transparency.